

ResNet

Authors: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research)

Published: CVPR 2016 (arXiv:1512.03385v1, Dec 2015)

Achievement: ILSVRC 2015 1st place (Classification, Detection, Localization), COCO 2015 1st place (Detection, Segmentation)

Abstract

Main Contribution

- **Deep network의 학습 난이도 문제:** 네트워크가 깊어질수록 학습이 어려워지는 문제를 해결
- **Residual Learning Framework** 제안
 - 레이어들이 unreferenced function을 학습하는 대신 **residual function**을 학습하도록 재구성
 - Input을 참조하여 학습

Key Results

- **ImageNet:** 152-layer ResNet 달성
 - VGG nets보다 8배 깊지만 복잡도는 더 낮음
 - Test error: 3.57% (ensemble)
 - ILSVRC 2015 classification 1위
- **CIFAR-10:** 100층, 1000층 네트워크 실험 성공
- **Object Detection:** COCO dataset에서 28% 상대적 성능 향상
- **Multiple 1st places:** ImageNet detection, ImageNet localization, COCO detection, COCO segmentation

Core Insight

Representation의 깊이가 많은 visual recognition task에서 핵심적으로 중요함

1. Introduction

Background: Deep Network의 중요성

- Deep convolutional neural networks가 image classification에서 breakthrough 달성
- Deep network는 low/mid/high-level features와 classifiers를 end-to-end multi-layer 방식으로 자연스럽게 통합
- Feature의 "levels"는 stacked layers의 수(depth)로 풍부해질 수 있음
- Network depth가 매우 중요함이 입증됨 (VGG: 16-19 layers, others: up to 30 layers)

Research Question

| Learning better networks가 단순히 더 많은 layer를 쌓는 것만큼 쉬운가?

Problem 1: Vanishing/Exploding Gradients

- 초기의 악명 높은 문제
- 해결됨:
 - Normalized initialization
 - Intermediate normalization layers (Batch Normalization)
 - 수십 개 layer를 가진 네트워크가 SGD + backpropagation으로 수렴 가능

Problem 2: Degradation Problem

- **현상:** Network depth가 증가하면 accuracy가 saturation되고 급격히 degraded됨
- **놀라운 점:** Overfitting이 아님!
 - 더 깊은 모델이 **training error**도 더 높음

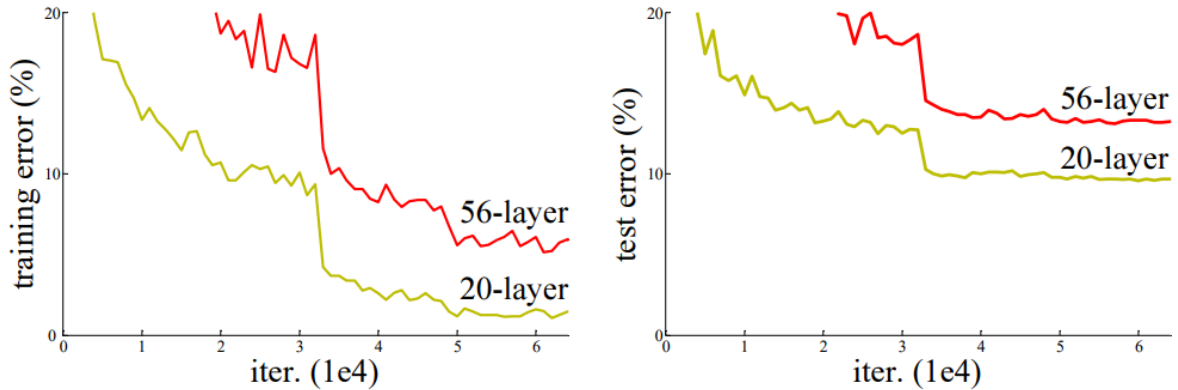


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

- 56-layer plain network가 20-layer보다 training/test error 모두 높음

Degradation Problem의 이론적 배경

- **Construction solution 존재:**
 - Shallow architecture가 있고 거기에 레이어를 추가한 deeper model을 생각해보면
 - 추가된 레이어들을 identity mapping으로, 나머지는 shallow model에서 복사하면
 - Deeper model이 shallow model보다 training error가 높으면 안 됨
- **But:** 현재 solver들은 이런 solution 을 찾지 못함

Proposed Solution: Deep Residual Learning

Core Idea

- Desired underlying mapping을 $H(x)$ 라고 할 때
- Stacked layers가 $H(x)$ 를 직접 fit하는 대신
- **Residual mapping** $F(x) := H(x) - x$ 를 fit하도록 함
- Original mapping은 $F(x) + x$ 로 재구성

Hypothesis

Residual mapping을 optimize하는 것이 original, unreferenced mapping을 optimize하는 것보다 쉬움

Extreme Case

- Identity mapping이 optimal이라면
- Residual을 zero로 push하는 것이 nonlinear layer stack으로 identity를 fit하는 것보다 쉬움

Implementation

- **Shortcut connections** 사용 (Figure 2)
 - 하나 이상의 layer를 skip
 - Identity mapping을 수행하고 output에 element-wise addition
 - **Extra parameter 없음**
 - **Computational complexity 증가 없음**
- End-to-end training 가능 (SGD + backpropagation)
- Caffe 등 common library로 쉽게 구현 가능

Experimental Results Summary

ImageNet

1. Extremely deep ResNets는 optimize하기 쉬움

- Plain nets는 depth 증가 시 higher training error
- ResNets는 depth 증가로부터 accuracy gain

2. 152-layer ResNet:

- ImageNet에서 가장 깊은 네트워크
- VGG보다 여전히 낮은 complexity
- Ensemble: 3.57% top-5 error (ILSVRC 2015 classification 1위)

CIFAR-10

- 100 layers, 1000 layers 모델 실험
- Optimization difficulties와 method의 효과가 특정 dataset에만 국한되지 않음을 시사

Other Tasks

- **COCO object detection:** 28% relative improvement
- **ILSVRC & COCO 2015:**
 - ImageNet detection 1위
 - ImageNet localization 1위
 - COCO detection 1위
 - COCO segmentation 1위

Importance of Depth

Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset.

2. Related Work

2.1 Residual Representations

Image Recognition

- **VLAD:** Dictionary에 대한 residual vectors로 encoding하는 representation
- **Fisher Vector:** VLAD의 probabilistic version으로 formulation 가능
- 둘 다 image retrieval과 classification을 위한 powerful shallow representations
- **Vector quantization:** Residual vectors를 encoding하는 것이 original vectors를 encoding하는 것보다 효과적

Low-level Vision & Computer Graphics

- **Multigrid method** (PDEs 해결):
 - System을 multiple scales의 subproblems로 reformulate
 - 각 subproblem은 coarser와 finer scale 사이의 residual solution을 담당
- **Hierarchical basis preconditioning:**
 - Alternative to Multigrid
 - Two scales 사이의 residual vectors를 나타내는 variables에 의존

- 이들 solver의 특징: Standard solvers보다 훨씬 빠르게 수렴
 - Residual nature를 인식하지 못하는 solver보다 효율적

Insight

| Good reformulation 또는 preconditioning이 optimization을 simplify할 수 있음

2.2 Shortcut Connections

Historical Background

- Shortcut connections의 practices와 theories는 오래 전부터 연구됨

Early Practice

- **Multi-layer perceptrons (MLPs) 학습:**
 - Network input에서 output으로 연결되는 linear layer 추가

Addressing Vanishing/Exploding Gradients

- 몇몇 intermediate layers를 auxiliary classifiers에 직접 연결
- Layer responses, gradients, propagated errors를 centering하는 방법들 (shortcut connections로 구현)

Inception Layer

- Shortcut branch와 몇 개의 deeper branches로 구성

2.3 Highway Networks

Concurrent Work

- ResNet과 동시대에 제안됨
- **Gating functions**를 가진 shortcut connections 제시

ResNet과의 차이점

Aspect	Highway Networks	ResNets (Ours)
Gates	Data-dependent	Parameter-free

Aspect	Highway Networks	ResNets (Ours)
Parameters	추가 parameter 있음	Extra parameter 없음
Shortcut behavior	"Closed" 가능 (approaching zero)	Never closed
Function type	Non-residual functions (when closed)	Always residual functions
Information flow	Gated	Always passed through

Key Distinction

- Highway: Gated shortcut이 "closed"되면 non-residual functions 표현
- ResNet:
 - Identity shortcuts는 절대 closed되지 않음
 - 모든 정보가 항상 통과
 - Additional residual functions를 학습

Performance Comparison

- Highway networks는 extremely increased depth (예: 100+ layers)에서 accuracy gain을 입증하지 못함
- ResNet은 100+ layers에서도 명확한 accuracy gain 달성

Key Takeaways

Problem Definition

1. **Degradation problem:** 단순히 layer를 쌓으면 training error가 오히려 증가
2. Not caused by overfitting → optimization difficulty

Solution

1. **Residual learning:** $H(x)$ 대신 $F(x) = H(x) - x$ 학습
2. **Identity shortcuts:** Parameter-free, always open
3. Easy to optimize, enables extremely deep networks (100+ layers)

Impact

1. State-of-the-art results across multiple tasks
2. Proven that depth matters significantly

- 3. Generic principle applicable to various vision and non-vision problems

3. Deep Residual Learning

3.1 Residual Learning

Problem Reformulation

- 기존 네트워크는 몇 개의 stacked layer가 underlying mapping $H(x)H(x)H(x)$ 를 직접 학습한다고 가정
- x : 해당 블록의 입력
- $H(x)$: 우리가 근사하려는 함수

Core Idea

- stacked layers가 $H(x)H(x)H(x)$ 대신 **residual function**을 학습하도록 재정의

$$F(x) := H(x) - x$$

$$H(x) = F(x) + x$$

Theoretical Equivalence

- 여러 비선형 레이어가 복잡한 함수를 근사할 수 있다면,
- residual function 또한 동일하게 근사 가능
- 즉 **표현력 측면에서는 동일**

Optimization Motivation

Degradation Problem

- depth 증가 시 training error 증가
- 이는 overfitting 때문이 아님
- deeper model은 shallower model의 superset이므로, 최소한 같은 training error 가능해야 함
- 그러나 실제 solver는 identity mapping을 학습하지 못함

Identity Mapping Argument

- 만약 최적 함수가 identity mapping이라면:

기존 방식:

- 여러 nonlinear layer가 identity를 직접 근사해야 함

Residual 방식:

- $F(x)=0, F(x)=0, F(x)=0$ 으로 만들면 됨
- weight를 0에 수렴시키는 것이 훨씬 쉬움

Preconditioning Interpretation

- residual reformulation은 optimization problem을 precondition
- identity mapping을 기준으로 두고 perturbation만 학습

Empirical Observation (후반 실험에서 언급)

- residual function의 response magnitude가 작음
- 실제 학습 결과가 identity에 대한 작은 수정 형태

3.2 Identity Mapping by Shortcuts

Residual Block Definition

$$y = F(x, W_i) + x$$

- x : input
- y : output
- F : residual mapping
- W_i : weight 집합

Example (2-layer residual function)

$$F = W_2 \sigma(W_1 x)$$

- σ : ReLU
- bias는 notation 단순화를 위해 생략

Shortcut Connection

- $+x$ 는 shortcut connection으로 구현
- element-wise addition 수행

Key Properties

- shortcut은 identity mapping

- 추가 parameter 없음
 - 추가 연산 거의 없음
 - SGD + backprop으로 end-to-end 학습 가능
 - Caffe 등 기존 라이브러리에서 구현 가능
-

Dimension Matching Issue

Residual addition을 위해:

- input과 output dimension 동일해야 함
-

Two Options for Dimension Increase

Option A — Identity + Zero Padding

- shortcut은 그대로 identity
 - 부족한 채널은 zero padding
 - 추가 parameter 없음
-

Option B — Projection Shortcut

$$y = F(x) + W_s x$$

- W_s : 1×1 convolution
 - stride=2로 downsampling 가능
 - dimension 정렬 목적
-

Experimental Finding

- identity shortcut만으로 degradation 문제 해결 가능
 - projection shortcut은 필수 아님
 - dimension mismatch 시에만 사용
-

Residual Function Flexibility

- F는 2-layer 또는 3-layer 가능
- 1-layer일 경우 linear layer와 유사 → 장점 관찰되지 않음
- fully-connected 뿐 아니라 convolution layer에도 동일 적용

- element-wise addition은 feature map channel 단위 수행

3.3 Network Architectures

Plain Network (Baseline)

Design Philosophy

- VGG 스타일 기반
 - 대부분 3×3 convolution
 - 두 가지 설계 원칙:
 - 같은 feature map size → 같은 filter 수
 - feature map size 절반 → filter 수 2배
-

Downsampling

- stride=2 convolution으로 수행
 - 마지막은 global average pooling + 1000-way FC + softmax
-

34-layer Plain Model

- 3.6B FLOPs
 - VGG-19의 18% 연산량
 - degradation 문제 발생
-

Residual Network Construction

- plain network에 shortcut 삽입
 - 동일 dimension → identity shortcut
 - dimension 증가 → option A or B
-

Bottleneck Architecture (Deep Models)

Motivation

- 깊은 네트워크의 연산량 문제 해결

Structure

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

- 첫 1×1: dimension reduction
 - 3×3: main computation
 - 마지막 1×1: dimension restoration
-

Importance of Identity Shortcut in Bottleneck

- projection shortcut 사용 시
 - 계산량 2배 증가
 - model size 증가
 - identity shortcut 유지가 효율성 유지 핵심
-

Deeper Networks

- 50-layer ResNet
- 101-layer ResNet
- 152-layer ResNet

152-layer:

- 11.3B FLOPs
- VGG-16/19보다 연산량 적음
- degradation problem 없음

3.4 Implementation

ImageNet Training Setting

- SGD
- mini-batch size 256
- learning rate 0.1
- plateau 시 10배 감소
- 최대 60×10^4 iterations
- weight decay 0.0001
- momentum 0.9

- dropout 사용 안함
 - Batch Normalization 사용
-

Batch Normalization Placement

- convolution 뒤
 - activation 앞
-

Data Augmentation

- short side $\in [256, 480]$
 - random 224×224 crop
 - horizontal flip
 - per-pixel mean subtraction
 - color augmentation
-

Testing Protocol

- standard 10-crop testing
 - fully convolutional evaluation
 - multi-scale testing:
 - short side $\in \{224, 256, 384, 480, 640\}$
 - score 평균
-

Detection Fine-tuning에서의 BN 처리

- ImageNet에서 BN statistics 계산
- detection fine-tuning 시 BN 고정
- 메모리 사용 감소 목적

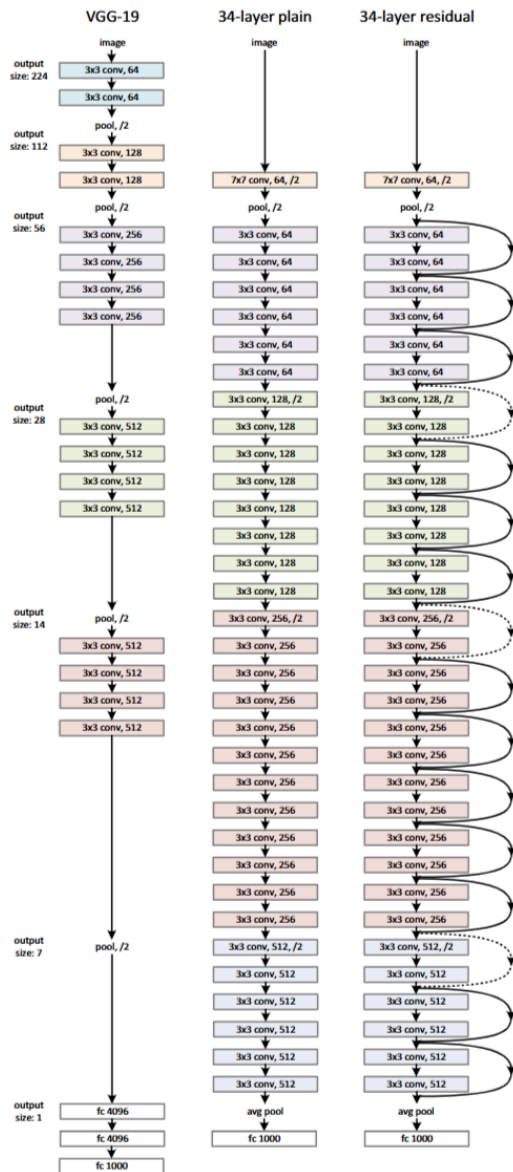
4. Experiments

4.1. ImageNet Classification

실험 공통 설정 (ImageNet)

- 데이터: ImageNet 2012 (1000 classes)
 - Train: 1.28M / Val: 50k / Test: 100k
 - 지표: Top-1 error, Top-5 error
 - 학습:
 - SGD, batch size 256
 - lr 0.1 시작 → plateau 시 10배 감소
 - 최대 60×10^4 iterations
 - weight decay $1e-4$, momentum 0.9
 - BN 사용, dropout 없음
 - 테스트:
 - 비교 실험: 10-crop
 - best 결과: fully-conv + multi-scale average (shorter side $\in \{224, 256, 384, 480, 640\}$)
-

4.1.1. Plain Networks



Left : the VGG-19 model [41] (19.6 billion FLOPs) as a reference.

Middle : a plain network with 34 parameter layers (3.6 billion FLOPs).

Right : a residual network with 34 parameter layers (3.6 billion FLOPs).

목적

- "깊게 쌓으면 더 좋아질까?"에 대한 반례를 실험으로 보여줌.
- 즉, **degradation problem**(깊어질수록 성능이 아니라 학습 자체가 망가짐)을 ImageNet에서도 검증.

실험 구성

- **plain-18 vs plain-34** 비교
 - 🤔 **plain network** : residual 구조 없이 단순히 층을 쌓은 구조

- VGG 스타일처럼 단순히 3×3 conv를 깊게 쌓은 구조
- ResNet과 비교하기 위해 파라미터 수/연산량 구조를 최대한 동일하게 설계

결과

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2

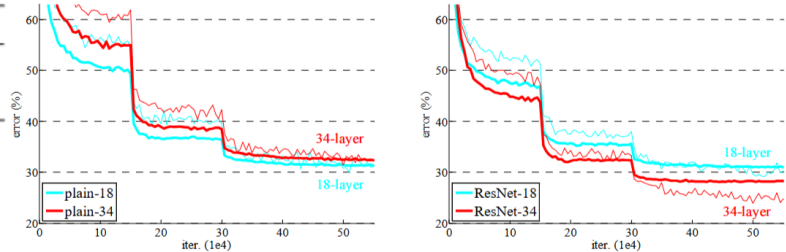


Figure 4

- validation top-1 error (10-crop):
 - plain-18: **27.94**
 - plain-34: **28.54** (더 깊은데 더 나쁨)
- 학습 과정(Fig.4):
 - plain-34가 **training error도 더 높음**
 - 즉, 단순히 층을 깊게 만든다고 ImageNet 성능이 좋아지지 않는다.
 - 즉 "overfitting" 때문이 아니라 **optimization 실패**

해석

- BN(Batch Normalization)을 써서 vanishing gradient 문제는 해결된 상태인데도,
 - 깊어지면 training error가 줄어들지 않는 현상이 발생.
- 저자들은 "깊은 plain net은 수렴 속도가 지수적으로 느려질 수 있다"는 가설을 제시함.

4.1.2. Residual Networks

목적

residual learning이 degradation problem을 해결하는지 확인.

실험 구성

- ResNet-18 vs ResNet-34
- shortcut은 **identity + zero padding(option A)** 사용

→ 즉, **파라미터 추가 없이** plain과 동일한 조건

결과

	plain	ResNet
18 layers	27.94	27.88
34 layers	28.54	25.03

Table 2

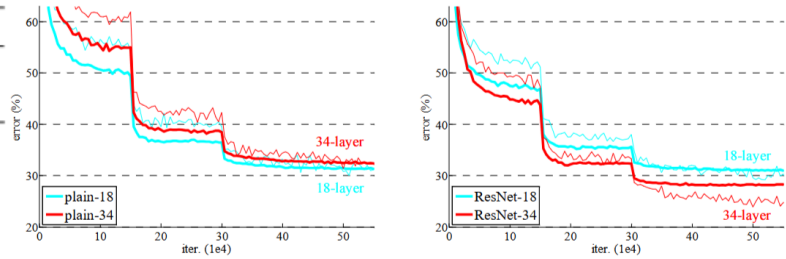


Figure 4

- validation top-1 error:
 - ResNet-18: **27.88**
 - ResNet-34: **25.03** (깊어질수록 좋아짐)
- Fig.4:
 - ResNet-34는 training error가 훨씬 낮고
 - validation도 같이 개선됨 → 일반화도 됨

해석

1. **plain과 반대로** ResNet은 깊어질수록 좋아짐
2. ResNet-34는 plain-34보다 training error가 확실히 낮아짐 → **optimization 개선**
3. 18-layer에서는 plain과 ResNet 정확도가 비슷하지만, ResNet이 더 빠르게 수렴

4.1.3. Identity vs. Projection Shortcuts

목적

| shortcut을 identity로만 써도 충분한지, projection이 꼭 필요한지 비교.

shortcut 옵션 3개

- **A:** 차원 증가 시 zero-padding (파라미터 없음)
- **B:** 차원 증가에서만 projection (1×1 conv)
- **C:** 모든 shortcut이 projection

결과 (ImageNet val, 10-crop)

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PRelu-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

- plain-34 : 28.54 / 10.02
- ResNet-34 A : 25.03 / 7.76
- ResNet-34 B : 24.52 / 7.46
- ResNet-34 C : 24.19 / 7.40

해석

- A/B/C 모두 plain보다 압도적으로 좋음 → residual 구조가 핵심
- B가 A보다 약간 좋은 이유:
 - A는 **zero padded dimension**에서는 **residual 학습이 사실상 없음**
- C는 B보다 아주 조금 더 좋지만,
 - projection shortcut이 많아지며 **파라미터가 늘어서 얻는 이득일 가능성이 큼**
- 결론:
 - degradation 해결에는 **projection이 필수 아님.**
 - 이후 실험에서는 비용 때문에 C는 버리고, B를 주로 사용

4.1.4. Deeper Bottleneck Architectures

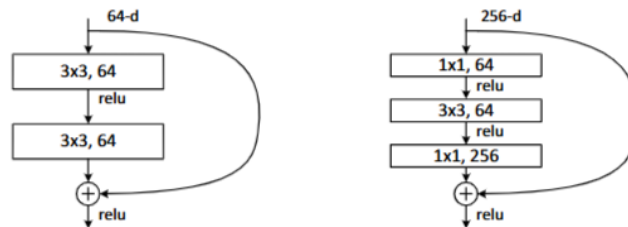
layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

목적

ResNet을 50/101/152까지 깊게 만들기 위해, 연산량과 파라미터 폭발을 막는 구조가 필요.

bottleneck block

- 기존 ResNet-34는 2-layer block ($3 \times 3 + 3 \times 3$)
- ResNet-50 이상은 3-layer bottleneck:



- 1×1 (차원 줄임)
- 3×3 (작은 차원에서 처리)
- 1×1 (차원 복원)

bottleneck에서 identity shortcut이 중요한 이유

- 만약 shortcut도 projection이면,
 - shortcut이 high-dim 끝단을 연결하게 되어
 - 연산량과 모델 크기가 거의 2배가 됨
- 즉 bottleneck + identity shortcut 조합이 효율의 핵심

4.1.5. 50-layer ResNet

구성

- ResNet-34의 2-layer block을 bottleneck 3-layer block으로 교체
- shortcut은 **option B**(차원 증가에만 projection)
- FLOPs: 3.8×10^9

성능 (ImageNet val, 10-crop)

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PRReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

- ResNet-50: top-1 **22.85**, top-5 **6.71**

4.1.6. 101-layer and 152-layer ResNets

구성

- Table 1에서 conv4_x 블록 수를 크게 늘려 깊이를 확장
- FLOPs:
 - ResNet-101: 7.6×10^9
 - ResNet-152: 11.3×10^9
- 흥미로운 점:
 - ResNet-152는 깊이가 엄청난데도 VGG-16/19보다 FLOPs가 낮음

성능 (ImageNet val, 10-crop)

model	top-1 err.	top-5 err.
VGG-16 [41]	28.07	9.33
GoogLeNet [44]	-	9.15
PReLU-net [13]	24.27	7.38
plain-34	28.54	10.02
ResNet-34 A	25.03	7.76
ResNet-34 B	24.52	7.46
ResNet-34 C	24.19	7.40
ResNet-50	22.85	6.71
ResNet-101	21.75	6.05
ResNet-152	21.43	5.71

Table 3. Error rates (% , 10-crop testing) on ImageNet validation. VGG-16 is based on our test. ResNet-50/101/152 are of option B that only uses projections for increasing dimensions.

- ResNet-101: top-1 **21.75**, top-5 **6.05**
- ResNet-152: top-1 **21.43**, top-5 **5.71**

성능 (ImageNet val, single-model; Table 4)

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of single-model results on the ImageNet validation set (except [†] reported on the test set).

- ResNet-101: top-1 **19.87**, top-5 **4.60**
- ResNet-152: top-1 **19.38**, top-5 **4.49**

해석

- plain net에서 발생했던 degradation이 전혀 없음.
- 깊이를 늘리는 것이 “학습 가능”해졌고, 실제로 정확도가 계속 상승.

4.1.7. Comparisons with State-of-the-art Methods

Single model 비교 (Table 4)

method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of single-model results on the ImageNet validation set (except [†] reported on the test set).

- PReLU-net: top-5 **5.71**
- BN-inception: top-5 **5.81**
- ResNet-152: top-5 **4.49**
→ single model인데 기존 ensemble급을 이김

Ensemble 결과 (Table 5, test set)

method	top-5 err. (test)
VGG [41] (ILSVRC'14)	7.32
GoogLeNet [44] (ILSVRC'14)	6.66
VGG [41] (v5)	6.8
PReLU-net [13]	4.94
BN-inception [16]	4.82
ResNet (ILSVRC'15)	3.57

Table 5. Error rates (%) of ensembles. The top-5 error is on the test set of ImageNet and reported by the test server

- ResNet ensemble: top-5 **3.57%**
- 이 결과가 ILSVRC 2015 1위

4.2. CIFAR-10 and Analysis

CIFAR-10 공통 설정

- 데이터: 50k train / 10k test, 10 classes
- 입력: 32×32, mean subtraction
- 구조:
 - 3×3 conv 기반
 - feature map size: {32,16,8}
 - filter: {16,32,64}
 - 총 layer 수 = $6n + 2$
- shortcut은 항상 identity(option A) 사용
→ plain vs residual을 완전히 공정 비교

4.2.1. CIFAR 실험 결과(깊이 증가)

비교한 깊이

- $n=\{3,5,7,9\} \rightarrow 20,32,44,56$ layers
- 추가로 $n=18 \rightarrow 110$ layers

결과 (Table 6)

method			error (%)
Maxout [10]			9.38
NIN [25]			8.81
DSN [24]			8.22
	# layers	# params	
FitNet [35]	19	2.5M	8.39
Highway [42, 43]	19	2.3M	7.54 (7.72±0.16)
Highway [42, 43]	32	1.25M	8.80
ResNet	20	0.27M	8.75
ResNet	32	0.46M	7.51
ResNet	44	0.66M	7.17
ResNet	56	0.85M	6.97
ResNet	110	1.7M	6.43 (6.61±0.16)
ResNet	1202	19.4M	7.93

Table 6. Classification error on the CIFAR-10 test set. All methods are with data augmentation. For ResNet-110, we run it 5 times and show "best (mean±std)" as in [43].

- ResNet-20: **8.75%**
- ResNet-32: **7.51%**

- ResNet-44: **7.17%**
- ResNet-56: **6.97%**
- ResNet-110: **6.43%** (best; mean $\pm$ std도 제공)

plain net의 문제

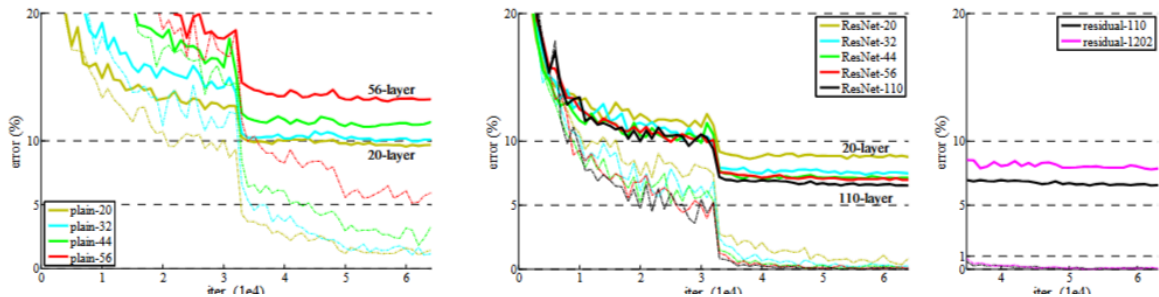


Figure 6. Training on CIFAR-10. Dashed lines denote training error, and bold lines denote testing error. Left: plain networks. The error of plain-110 is higher than 60% and not displayed.

Middle: ResNets.

Right: ResNets with 110 and 1202 layers

- Fig.6에서 plain-20 → 56으로 갈수록 training error가 커짐
- plain-110은 training error가 60% 넘어서 그래프에도 표시 안됨

ResNet은?

- Fig.6에서 ResNet은 깊어질수록 training error가 낮아지고 test도 좋아짐
- 즉 CIFAR에서도 degradation 해결

4.2.2. Analysis of Layer Responses

목적

- residual function $F(x)$ 의 출력이 실제로 “작은 perturbation”인지 실증적으로 확인.

분석 방식

- 각 3×3 layer output을 측정:
 - BN 이후
 - ReLU/addition 이전
- 각 layer response의 표준편차(std)를 계산

결과 (Fig.7)

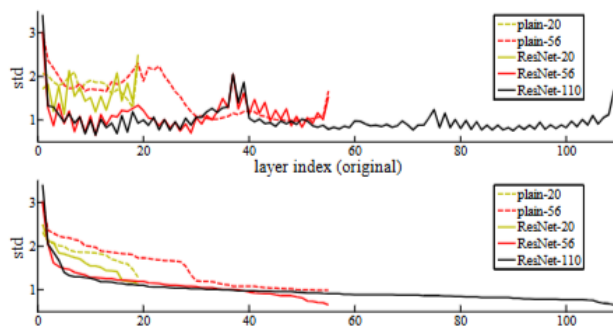


Figure 7. Standard deviations (std) of layer responses on CIFAR-10. The responses are the outputs of each 3×3 layer, after BN and before nonlinearity. Top: the layers are shown in their original order. Bottom: the responses are ranked in descending order

- ResNet 계열이 plain보다 response std가 전반적으로 작음.
- ResNet이 깊어질수록(20 → 56 → 110) response magnitude가 더 작아짐.

해석

- residual mapping이 실제로 0에 가까운 작은 수정이라는 가정이 맞았음을 뒷받침.
- 깊어질수록 "각 layer가 전체 신호를 크게 바꾸기보단 조금씩만 수정"하는 구조가 됨.

4.2.3. Exploring Over 1000 layers

목적

- "ResNet은 진짜 1000층도 학습 가능한가?" 확인.

실험

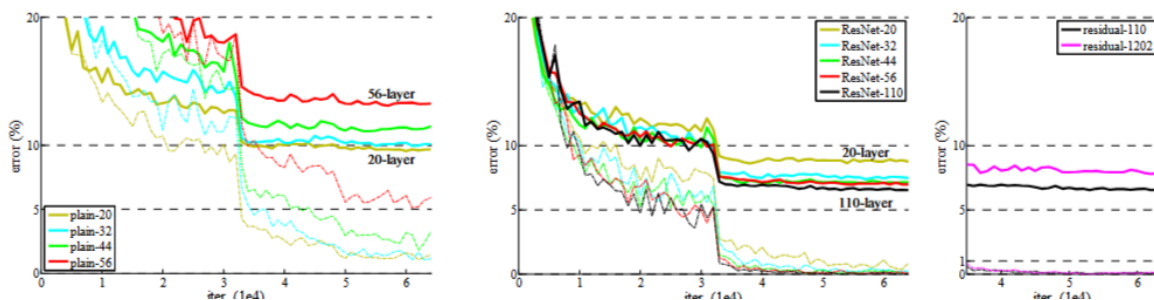


Figure 6. Training on CIFAR-10. Dashed lines denote training error, and bold lines denote testing error. Left: plain networks. The error of plain-110 is higher than 60% and not displayed.

Middle: ResNets.

Right: ResNets with 110 and 1202 layers

- n=200 → **1202-layer ResNet**
- Fig.6 (right):
 - training error < 0.1%까지 내려감
 - optimization 자체는 성공

성능

- test error: **7.93%**
- 110-layer보다 오히려 나쁨

해석

- optimization 문제는 해결됐지만, CIFAR처럼 작은 데이터에서는 1202-layer는 너무 큰 모델 (19.4M params)이라 overfitting 가능성이 큼.
- 저자들은 maxout/dropout 같은 강한 regularization을 결합하면 개선될 수 있다고 언급

4.3. Object Detection on PASCAL and MS COCO

실험 목적

- ResNet이 분류만 잘하는 게 아니라, detection에서도 representation을 크게 개선하는지 확인.

실험 설정

- detector: Faster R-CNN
- 비교: VGG-16 vs ResNet-101
- detection pipeline은 동일하게 유지
 - 성능 차이는 backbone feature 차이로만 설명 가능

PASCAL VOC 결과 (Table 7)

training data	07+12	07++12
test data	VOC 07 test	VOC 12 test
VGG-16	73.2	70.4
ResNet-101	76.4	73.8

Table 7. Object detection mAP (%) on the PASCAL VOC 2007/2012 test sets using baseline Faster R-CNN.
See also Table 10 and 11 for better results

데이터 세팅

- VOC07 test:
 - train = VOC07 trainval(5k) + VOC12 trainval(16k) = "07+12"
- VOC12 test:
 - train = VOC07 trainval+test(10k) + VOC12 trainval(16k) = "07++12"

성능 (mAP)

- VOC07 test:
 - VGG-16: **73.2**
 - ResNet-101: **76.4**
- VOC12 test:
 - VGG-16: **70.4**
 - ResNet-101: **73.8**

해석

- ResNet-101이 VOC에서 mAP를 3% 이상 끌어올림.
- detection 시스템이 같으므로 "feature representation이 좋아졌기 때문"이라고 결론.

MS COCO 결과 (Table 8)

metric	mAP@.5	mAP@[.5, .95]
VGG-16	41.5	21.2
ResNet-101	48.4	27.2

Table 8. Object detection mAP (%) on the COCO validation set using baseline Faster R-CNN. See also Table 9 for better results

평가 지표

- mAP@0.5 (VOC 방식)
- mAP@[0.5:0.95] (COCO 표준)

성능 (COCO val)

- VGG-16:
 - mAP@0.5 = **41.5**

- $\text{mAP@} [.5, .95] = 21.2$
- ResNet-101:
 - $\text{mAP@} 0.5 = 48.4$
 - $\text{mAP@} [.5, .95] = 27.2$

핵심 해석

- COCO 표준 지표($\text{mAP@} [.5, .95]$)에서 **+6.0%p** 증가
- 상대 개선율은 **28% 향상**
- 논문에서 강조하는 포인트:
 - 이 개선은 “오로지 backbone representation이 좋아져서” 생긴 것